

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf\_6c18d403-385\agent-aca7d38c0abffd0ac.jsonl`
- SHA-256: `3b42129294b98a55c5a9ed639a04cb595fbd8b990ca8ece4596a63e1c70cf992`
- Source modified: `2026-07-06T20:55:37+00:00`
- Imported at: `2026-07-08T16:00:07+00:00`
- project: `wf\_6c18d403-385`
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`

Transcript

1. user (2026-07-06T20:52:38.499Z)

Reviewing GitHub PR #50 of the repo at C:/Users/avidu/Projects/Telegram ? "feat(policy): add F1 pipeline primitives" (branch feat/f1-policy-engine -> main).

It is the first F1 slice of ADR 0012 (policy pipeline, just merged as #40). +280/-14, 4 files:

- src/tg_video_filter/policy.py (NEW): PolicyContext, StageResult (+ pass_/fail helpers), PolicyStage Protocol, evaluate_pipeline().
- src/tg_video_filter/db.py: NEW load_policy_config/save_policy_config (raw JSON dict over policies.config_json);
load_filter_config now uses model_validate(_policy_config_json_to_dict(...)) instead of model_validate_json;
save_filter_config now MERGES cfg.model_dump(mode='json') into the existing raw config dict (preserving unknown keys) instead of replacing the whole blob with cfg.model_dump_json().
- tests/test_db.py + tests/test_policy.py: primitives tests + a preserve-unknown-keys regression test.

A checked-out copy of the PR is at C:/Users/avidu/Projects/_wt-pr50 (read files there or via 'git -C C:/Users/avidu/Projects/Telegram show origin/feat/f1-policy-engine:<path>').

Run repros with: PYTHONPATH=C:/Users/avidu/Projects/_wt-pr50/src
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script> (build schema via db._apply_schema on aiosqlite ":memory:").

BACKGROUND ? why this PR exists: ADR 0012 review found that load_filter_config/save_filter_config would ERASE future pipeline keys (topics/security_checks/deep_inspect) when a user runs /set, because save re-serialized via FilterConfig (which drops unknown keys). F1 fixes that persistence boundary. The main reviewer already verified the happy path works (pipeline keys survive a /set; flat fields update). Your job: find edge cases / bugs / regressions the happy path missed.

Ground rules:

- READ actual code, cite file:line. Run repros. Authoritative gate ALREADY RUN: ruff clean, 501 passed @ 96.63%.
- Report ONLY defects in THIS PR's diff. No praise. Concrete failure scenario per finding. Rank most-severe first.
- Severity: high (real bug/regression/data-loss), medium (edge case, latent, asymmetry), low (typing/hygiene/scope-note).

DIMENSION: policy.py primitives correctness + ADR conformance + scope.

Investigate:

- evaluate_pipeline: short-circuit + order correctness (tests cover it) ? any subtle bug (e.g., does it run/accumulate the failing stage's result? yes it appends then returns ? confirm). Async correctness.

- StageResult is a frozen dataclass but metadata is a mutable dict (field(default_factory=dict)); pass_/fail helpers.
Any mutable-default or frozen-inconsistency issue?
- PolicyContext.fetcher/budget typed as 'object | None' (ADR named ContentFetcher/PolicyBudget)
? acceptable placeholder
or a typing gap?
- SCOPE: ADR 0012's F1 scope includes "hot-path wiring from live handler to ContentItem/Source/Policy context". This PR
does NOT touch telethon_client.py ? evaluate_pipeline/PolicyStage are not called anywhere except tests (dead until a
later slice). Is that disclosed (PR body says "first F1 slice")? Is policy.py currently unreachable code? Note it as a
scope observation with correct severity (low, if disclosed).
- Is policy.py exported/importable cleanly; __all__ complete; any circular import risk (policy imports models)?

Run repros. Report findings.

2. assistant (2026-07-06T20:52:42.621Z)

I'll investigate the policy.py primitives and the db.py changes. Let me start by reading the actual PR code.

3. user (2026-07-06T20:52:43.633Z)

```

1      """Policy pipeline primitives for ADR 0012.
2
3      The policy layer is deliberately small and framework-free in F1: stages receive
4      an immutable context, return a structured result, and the evaluator stops on
5      the first failure. Later Track F phases add concrete stages.
6      """
7
8      from __future__ import annotations
9
10     from dataclasses import dataclass, field
11     from typing import Protocol
12
13     from tg_video_filter.models import ContentItem, Policy, Source
14
15
16     @dataclass(frozen=True)
17     class PolicyContext:
18         """Immutable snapshot available to a policy stage."""
19
20         content_item: ContentItem
21         source: Source
22         policy: Policy
23         fetcher: object | None = None
24         budget: object | None = None
25
26
27     @dataclass(frozen=True)
28     class StageResult:
29         """Outcome of one policy stage."""
30
31         passed: bool
32         reason: str
33         stage_name: str
34         metadata: dict[str, object] = field(default_factory=dict)
35
36     @staticmethod
37     def pass_(stage_name: str, reason: str, **metadata: object) -> StageResult:
38         return StageResult(
39             passed=True,
40             reason=reason,
41             stage_name=stage_name,
```

```

42         metadata=metadata,
43     )
44
45     @staticmethod
46     def fail(stage_name: str, reason: str, **metadata: object) -> StageResult:
47         return StageResult(
48             passed=False,
49             reason=reason,
50             stage_name=stage_name,
51             metadata=metadata,
52         )
53
54
55     class PolicyStage(Protocol):
56         """A single stateless gate in the policy pipeline."""
57
58         async def evaluate(self, ctx: PolicyContext) -> StageResult:
59             """Evaluate ``ctx`` and return a stage result."""
60             ...
61
62
63     async def evaluate_pipeline(
64         ctx: PolicyContext,
65         stages: list[PolicyStage],
66     ) -> tuple[bool, list[StageResult]]:
67         """Evaluate stages in order and short-circuit on the first failure."""
68         results: list[StageResult] = []
69         for stage in stages:
70             result = await stage.evaluate(ctx)
71             results.append(result)
72             if not result.passed:
73                 return False, results
74         return True, results
75
76
77     __all__ = [
78         "PolicyContext",
79         "PolicyStage",
80         "StageResult",
81         "evaluate_pipeline",
82     ]
83

```

4. user (2026-07-06T20:52:44.172Z)

<system-reminder>[Truncated: PARTIAL view ? showing lines 1-1389 of 1872 total (28619 tokens, cap 25000). Call Read with offset=1390 limit=1389 for the next page, or Grep to find a specific section. Do NOT answer from this page alone if the answer may be further in the file.]</system-reminder>

```

1     """SQLite persistence layer (aiosqlite).
2
3     Schema, migrations, and repository functions. See ``docs/design.md`` §3 for
4     the legacy data model and ``docs/adr/0011-provider-neutral-domain-schema.md``
5     for the provider-neutral ``sources`` / ``content_items`` / ``policies`` /
6     ``destinations`` / ``routing_rules`` / ``delivery_attempts`` schema (ADR 0011).
7
8     The repository functions take an ``aiosqlite.Connection`` so tests can inject an
9     in-memory database (``aiosqlite.connect(":memory:")``).
10    """
11
12    from __future__ import annotations
13
14    import json
15    from collections.abc import Awaitable, Callable, Iterable

```

```

16     from datetime import UTC, datetime
17
18     import aiosqlite
19
20     from tg_video_filter.models import (
21         Account,
22         Channel,
23         ContentItem,
24         DeliveryAttempt,
25         DeliveryStatus,
26         Destination,
27         FilterConfig,
28         Policy,
29         RoutingRule,
30         Source,
31         TelegramSession,
32         User,
33         VideoRecord,
34     )
35
36     # --- DDL -----
37
38     _SCHEMA = """
39     CREATE TABLE IF NOT EXISTS accounts (
40         id            INTEGER PRIMARY KEY,
41         telegram_id   INTEGER UNIQUE,
42         provider      TEXT NOT NULL DEFAULT 'telegram',
43         display_name  TEXT,
44         created_at    TEXT NOT NULL
45     );
46
47     CREATE TABLE IF NOT EXISTS telegram_sessions (
48         account_id    INTEGER PRIMARY KEY REFERENCES accounts(id),
49         phone         TEXT,
50         session_path  TEXT NOT NULL,
51         enabled       INTEGER NOT NULL DEFAULT 1,
52         created_at    TEXT NOT NULL,
53         last_login_at TEXT
54     );
55
56     CREATE TABLE IF NOT EXISTS channels (
57         id            INTEGER PRIMARY KEY AUTOINCREMENT,
58         user_id       INTEGER NOT NULL REFERENCES accounts(id),
59         channel_id    INTEGER NOT NULL,
60         title         TEXT,
61         seen_first_at TEXT NOT NULL,
62         UNIQUE (user_id, channel_id)
63     );
64
65     CREATE TABLE IF NOT EXISTS videos (
66         id            INTEGER PRIMARY KEY AUTOINCREMENT,
67         user_id       INTEGER NOT NULL REFERENCES accounts(id),
68         channel_id    INTEGER NOT NULL,
69         message_id    INTEGER NOT NULL,
70         document_id   INTEGER,
71         duration_s    INTEGER,
72         width         INTEGER,
73         height        INTEGER,
74         size_bytes    INTEGER,
75         mime_type     TEXT,
76         matched       INTEGER NOT NULL DEFAULT 0,
77         forwarded     INTEGER NOT NULL DEFAULT 0,
78         seen_at       TEXT NOT NULL,
79         UNIQUE (user_id, channel_id, message_id)
80     );

```

```

81
82 CREATE TABLE IF NOT EXISTS filter_config (
83     user_id      INTEGER PRIMARY KEY REFERENCES accounts(id),
84     config_json  TEXT NOT NULL
85 );
86
87 -- Provider-neutral domain tables (ADR 0011, Track B). ``sources``,
88 -- ``content_items``, and ``policies`` are populated by migration 003 (and by
89 -- the shims below); ``destinations`` / ``routing_rules`` / ``delivery_attempts``
90 -- are created empty here so the schema is complete ? Track C (#28) implements
91 -- the delivery logic that populates them.
92
93 CREATE TABLE IF NOT EXISTS sources (
94     id            INTEGER PRIMARY KEY AUTOINCREMENT,
95     account_id    INTEGER NOT NULL REFERENCES accounts(id),
96     provider      TEXT NOT NULL DEFAULT 'telegram',
97     provider_source_id TEXT NOT NULL,
98     title         TEXT,
99     kind          TEXT NOT NULL DEFAULT 'channel',
100    watched       INTEGER NOT NULL DEFAULT 0,
101    seen_first_at  TEXT NOT NULL,
102    last_seen_at   TEXT,
103    UNIQUE (account_id, provider, provider_source_id)
104 );
105
106 CREATE TABLE IF NOT EXISTS content_items (
107     id            INTEGER PRIMARY KEY AUTOINCREMENT,
108     account_id    INTEGER NOT NULL REFERENCES accounts(id),
109     source_id     INTEGER NOT NULL REFERENCES sources(id),
110     provider      TEXT NOT NULL DEFAULT 'telegram',
111     provider_message_id TEXT NOT NULL,
112     content_type  TEXT NOT NULL DEFAULT 'video',
113     text         TEXT,
114     link_url      TEXT,
115     media_ref     TEXT,
116     document_id   TEXT,
117     duration_s    INTEGER,
118     width         INTEGER,
119     height        INTEGER,
120     size_bytes    INTEGER,
121     mime_type     TEXT,
122     dedup_key     TEXT NOT NULL,
123     matched       INTEGER NOT NULL DEFAULT 0,
124     seen_at       TEXT NOT NULL,
125     UNIQUE (account_id, source_id, provider_message_id),
126     UNIQUE (account_id, dedup_key)
127 );
128
129 CREATE TABLE IF NOT EXISTS policies (
130     id            INTEGER PRIMARY KEY AUTOINCREMENT,
131     account_id    INTEGER NOT NULL REFERENCES accounts(id),
132     name          TEXT NOT NULL,
133     config_json   TEXT NOT NULL,
134     is_default    INTEGER NOT NULL DEFAULT 0,
135     created_at    TEXT NOT NULL,
136     updated_at    TEXT NOT NULL,
137     UNIQUE (account_id, name)
138 );
139
140 -- ADR 0011: exactly one default policy per account (structural invariant).
141 CREATE UNIQUE INDEX IF NOT EXISTS idx_policies_account_default
142 ON policies(account_id) WHERE is_default = 1;
143
144 CREATE TABLE IF NOT EXISTS destinations (
145     id            INTEGER PRIMARY KEY AUTOINCREMENT,

```

```

146     account_id  INTEGER NOT NULL REFERENCES accounts(id),
147     provider    TEXT NOT NULL DEFAULT 'telegram',
148     target      TEXT NOT NULL,
149     kind        TEXT NOT NULL DEFAULT 'saved',
150     title       TEXT,
151     is_output    INTEGER NOT NULL DEFAULT 0,
152     created_at  TEXT NOT NULL,
153     UNIQUE (account_id, provider, target)
154 );
155
156 CREATE TABLE IF NOT EXISTS routing_rules (
157     id           INTEGER PRIMARY KEY AUTOINCREMENT,
158     account_id   INTEGER NOT NULL REFERENCES accounts(id),
159     source_id    INTEGER NOT NULL REFERENCES sources(id),
160     policy_id    INTEGER REFERENCES policies(id),
161     destination_id INTEGER NOT NULL REFERENCES destinations(id),
162     enabled      INTEGER NOT NULL DEFAULT 1,
163     priority     INTEGER NOT NULL DEFAULT 0,
164     created_at   TEXT NOT NULL,
165     UNIQUE (account_id, source_id, destination_id)
166 );
167
168 CREATE TABLE IF NOT EXISTS delivery_attempts (
169     id           INTEGER PRIMARY KEY AUTOINCREMENT,
170     content_item_id INTEGER NOT NULL REFERENCES content_items(id),
171     destination_id INTEGER NOT NULL REFERENCES destinations(id),
172     route_id     INTEGER REFERENCES routing_rules(id),
173     status       TEXT NOT NULL DEFAULT 'pending',
174     attempt_count INTEGER NOT NULL DEFAULT 0,
175     last_error   TEXT,
176     created_at   TEXT NOT NULL,
177     delivered_at TEXT,
178     next_retry_at TEXT,
179     UNIQUE (content_item_id, destination_id)
180 );
181 ""
182
183 # Note: the ``idx_videos_user_document`` partial unique index is created by
184 # migration 001 (below), not in ``_SCHEMA``. It references ``document_id``
185 # which is present on fresh tables but only added to legacy v1 DBs by the
186 # migration; putting it in ``_SCHEMA`` would break ``executescript`` on legacy
187 # DBs that don't yet have the column.
188
189
190 async def init_db(db_path: str) -> None:
191     """Open the database at ``db_path`` and create/migrate the schema.
192
193     Safe to call on every startup. Applies the base schema (``IF NOT EXISTS``)
194     then runs any pending migrations via the ``user_version`` ladder.
195     """
196     async with aiosqlite.connect(db_path) as conn:
197         await _apply_schema(conn)
198
199
200 async def _apply_schema(conn: aiosqlite.Connection) -> None:
201     # Enable WAL mode (concurrent reads+writes), foreign key enforcement,
202     # and a busy timeout so concurrent access waits instead of failing immediately.
203     await conn.execute("PRAGMA journal_mode = WAL")
204     await conn.execute("PRAGMA foreign_keys = ON")
205     await conn.execute("PRAGMA busy_timeout = 5000")
206     await conn.executescript(_SCHEMA)
207     # ADR 0011 housekeeping: ``_SCHEMA`` still creates the legacy
208     # ``channels`` / ``videos`` / ``filter_config`` tables (they're needed for
209     # migration 002's FK rebuild and migration 003's backfill on legacy DBs).
210     # Once migration 003 has run (signalled by ``channels_deprecated`` existing),

```

```

211     # any *empty* recreations of those legacy tables from a subsequent
212     # ``init_db`` are dropped here so they don't shadow the renamed copies.
213     if await _table_exists(conn, "channels_deprecated"):
214         await conn.execute("DROP TABLE IF EXISTS channels")
215         await conn.execute("DROP TABLE IF EXISTS videos")
216         await conn.execute("DROP TABLE IF EXISTS filter_config")
217     await _apply_migrations(conn)
218     await conn.commit()
219
220
221     # --- migrations -----
222     #
223     # Migrations are applied in order via SQLite's ``PRAGMA user_version`` counter.
224     # Each migration is idempotent and bumps the version by 1. ``_SCHEMA`` above is
225     # the source of truth for *fresh* databases; the migrations below reconcile
226     # *existing* databases that were created by an older schema version.
227     #
228     # To add a new migration:
229     #   1. Append a ``(version, _migration_NN)`` tuple to ``_MIGRATIONS``.
230     #   2. Implement ``async def _migration_NN(conn) -> None`` (must be idempotent).
231     #   3. Add a test in ``tests/test_db.py`` that seeds an old schema, runs
232     #       ``init_db``, and asserts the new structure.
233
234     _MIGRATIONS: list[tuple[int, Callable[[aiosqlite.Connection], Awaitable[None]]]] = []
235
236
237     def _migration(
238         version: int,
239     ) -> Callable[
240         [Callable[[aiosqlite.Connection], Awaitable[None]]],
241         Callable[[aiosqlite.Connection], Awaitable[None]],
242     ]:
243         """Register a migration that brings the DB up to ``user_version == version``."""
244
245         def decorator(
246             fn: Callable[[aiosqlite.Connection], Awaitable[None]],
247         ) -> Callable[[aiosqlite.Connection], Awaitable[None]]:
248             _MIGRATIONS.append((version, fn))
249             return fn
250
251         return decorator
252
253
254     # --- provider-neutral dedup key (ADR 0011, Option 1A) -----
255     #
256     # Pure function: computes the ``content_items.dedup_key`` value from metadata
257     # already extracted at ingestion time (no download ? honors ADR 0003). The
258     # decision tree is ordered strongest-key first; each branch is guarded so
259     # partial metadata can never collapse to a degenerate key (e.g. ``tg:fp:0:0:0:0``).
260     # See ``compute_dedup_key_test`` in ``tests/test_db.py`` for the full matrix.
261
262
263     def compute_dedup_key(
264         *,
265         provider: str,
266         document_id: str | None = None,
267         source_id: int | str | None = None,
268         provider_message_id: str | None = None,
269         size_bytes: int | None = None,
270         duration_s: int | None = None,
271         height: int | None = None,
272         width: int | None = None,
273     ) -> str:
274         """Compute the provider-neutral ``dedup_key`` for a ``ContentItem``.
275

```

```

276 Implements the ADR 0011 Option 1A decision tree:
277
278 - **Telegram** (`provider == "telegram"`):
279     1. If `document_id` is set ? `tg:doc:{document_id}`.
280     2. Else if **all four** fingerprint fields (size, duration, height, width)
281        are set ? `tg:fp:{size}:{dur}:{h}:{w}`.
282     3. Else ? `tg:msg:{source_id}:{provider_message_id}` (Tier-0 fallback;
283        `source_id` is required because Telegram `message_id` is only
284        unique within a channel).
285 - **Other providers** (`provider != "telegram"`): fall back to the Tier-0
286    `{provider}:msg:{source_id}:{provider_message_id}` shape. Future
287    providers add their own branches (e.g. `wa:sha256:{blob_hash}`) here.
288
289 Raises `ValueError` if the fallback branch is reached without the fields
290 it requires (`source_id` + `provider_message_id`) ? a degenerate
291 `dedup_key` is worse than a loud failure at ingestion time.
292 """
293 p = provider.lower()
294 if p == "telegram":
295     if document_id is not None and document_id != "":
296         return f"tg:doc:{document_id}"
297     # All four fingerprint fields are required to avoid a degenerate
298     # `tg:fp:0:0:0:0` key when some metadata is missing.
299     if None not in (size_bytes, duration_s, height, width):
300         return f"tg:fp:{size_bytes}:{duration_s}:{height}:{width}"
301     # Tier-0 fallback.
302     if source_id is None or provider_message_id is None:
303         raise ValueError(
304             "Telegram dedup_key fallback requires source_id and provider_message_id"
305             "when document_id and the full fingerprint are unavailable"
306         )
307     return f"tg:msg:{source_id}:{provider_message_id}"
308 # Future-proof default for providers without a dedicated branch.
309 if source_id is None or provider_message_id is None:
310     raise ValueError(
311         f"dedup_key for provider={provider!r} requires source_id and
312         provider_message_id"
313     )
314 return f"{p}:msg:{source_id}:{provider_message_id}"
315
316 async def _apply_migrations(conn: aiosqlite.Connection) -> None:
317     """Run every registered migration whose version > current `user_version`."""
318     async with conn.execute("PRAGMA user_version") as cur:
319         row = await cur.fetchone()
320         current = int(row[0]) if row else 0
321
322     for version, fn in _MIGRATIONS:
323         if version <= current:
324             continue
325         await fn(conn)
326         await conn.execute(f"PRAGMA user_version = {version}")
327
328
329 async def _column_exists(conn: aiosqlite.Connection, table: str, column: str) -> bool:
330     """Return True if `column` exists on `table` (async, PRAGMA table_info)."""
331     async with conn.execute(f"PRAGMA table_info({table})") as cur:
332         rows = await cur.fetchall()
333         return any(row[1] == column for row in rows)
334
335
336 @migration(1)
337 async def _migration_001_add_document_id(conn: aiosqlite.Connection) -> None:
338     """Add `document_id` column + partial unique index (ADR 0006 content dedup).

```



```

339
340 Idempotent: safe on fresh DBs (column/index already present) and on v1 DBs
341 created before this migration existed.
342 """
343 if not await _column_exists(conn, "videos", "document_id"):
344     await conn.execute("ALTER TABLE videos ADD COLUMN document_id INTEGER")
345 # Create the partial unique index for both fresh and migrated DBs. Uses
346 # IF NOT EXISTS so it is safe on DBs where it was already created.
347 await conn.execute(
348     "CREATE UNIQUE INDEX IF NOT EXISTS idx_videos_user_document "
349     "ON videos(user_id, document_id) WHERE document_id IS NOT NULL"
350 )
351
352
353 @_migration(2)
354 async def _migration_002_identity_split(conn: aiosqlite.Connection) -> None:
355     """Split the legacy ``users`` table into ``accounts`` + ``telegram_sessions``.
356
357     ADR 0010. Integer-preserving: ``accounts.id`` copies ``users.id`` and the
358     data in ``channels`` / ``videos`` / ``filter_config`` is copied verbatim,
359     so their integer ``user_id`` values still resolve to the same identity (now
360     via ``accounts.id``). The legacy ``users`` table is renamed to
361     ``users_deprecated`` and kept for one release as a rollback safety net; a
362     follow-up migration drops it.
363
364     FK contract: ``channels`` / ``videos`` / ``filter_config`` are rebuilt with
365     ``REFERENCES accounts(id)`` so migrated DBs end up with the *same* FK shape
366     as fresh DBs. (A naive rename of ``users`` would leave SQLite rewriting
367     those FKs to point at ``users_deprecated``, diverging from fresh installs
368     and breaking inserts for new accounts ? see PR #33 review.)
369
370     Idempotent: a no-op once the legacy ``users`` table has been renamed.
371     """
372     # If the legacy users table is gone, this migration has already run
373     # (or the DB is fresh and only has accounts). Nothing to do.
374     if not await _table_exists(conn, "users"):
375         return
376
377     # Build accounts from the legacy users table. accounts.id := users.id.
378     # (The empty accounts table created by _SCHEMA on a legacy DB is dropped
379     # and recreated here so the INSERT can populate it cleanly.)
380     await conn.execute("DROP TABLE IF EXISTS accounts")
381     await conn.execute(
382         """
383         CREATE TABLE accounts (
384             id            INTEGER PRIMARY KEY,
385             telegram_id   INTEGER UNIQUE,
386             provider      TEXT NOT NULL DEFAULT 'telegram',
387             display_name  TEXT,
388             created_at    TEXT NOT NULL
389         )
390         """
391     )
392     await conn.execute(
393         """
394         INSERT INTO accounts (id, telegram_id, provider, created_at)
395         SELECT id, id, 'telegram', created_at FROM users
396         """
397     )
398
399     # Build telegram_sessions, one row per legacy userbot user.
400     # (Same drop-and-recreate as accounts: _SCHEMA created an empty copy.)
401     await conn.execute("DROP TABLE IF EXISTS telegram_sessions")
402     await conn.execute(
403         """

```

```

404     CREATE TABLE telegram_sessions (
405         account_id    INTEGER PRIMARY KEY REFERENCES accounts(id),
406         phone          TEXT,
407         session_path  TEXT NOT NULL,
408         enabled        INTEGER NOT NULL DEFAULT 1,
409         created_at     TEXT NOT NULL,
410         last_login_at TEXT
411     )
412     """"
413 )
414 # session_path is built from the *telegram_id* (= old users.id) to match
415 # the convention telethon_client.build_client used pre-migration. The
416 # sessions_dir prefix is prepended at read time (Settings.sessions_dir)
417 # so this migration does not need to know the runtime path.
418 await conn.execute(
419     """"
420     INSERT INTO telegram_sessions (account_id, phone, session_path, enabled,
created_at)
421     SELECT id, phone, 'user_' || id || '.session', enabled, created_at
422     FROM users
423     """"
424 )
425
426 # Rebuild channels / videos / filter_config so their FK constraints point
427 # at accounts(id) instead of the legacy users(id). SQLite's
428 # ALTER TABLE RENAME rewrites FK references in *other* tables to follow
429 # the rename ? so a naive rename of users ? users_deprecated would leave
430 # these tables referencing users_deprecated(id), diverging from fresh DBs
431 # (which reference accounts(id)) and breaking inserts for new accounts.
432 # The rebuild: rename the data-holding legacy table out of the way, create
433 # the new one with the correct FK, copy the data back, drop the renamed copy.
434 for table, new_ddl, copy_cols in (
435     (
436         "channels",
437         """"
438         CREATE TABLE channels (
439             id            INTEGER PRIMARY KEY AUTOINCREMENT,
440             user_id       INTEGER NOT NULL REFERENCES accounts(id),
441             channel_id    INTEGER NOT NULL,
442             title         TEXT,
443             seen_first_at TEXT NOT NULL,
444             UNIQUE (user_id, channel_id)
445         )
446         """,
447         "id, user_id, channel_id, title, seen_first_at",
448     ),
449     (
450         "videos",
451         """"
452         CREATE TABLE videos (
453             id            INTEGER PRIMARY KEY AUTOINCREMENT,
454             user_id       INTEGER NOT NULL REFERENCES accounts(id),
455             channel_id    INTEGER NOT NULL,
456             message_id    INTEGER NOT NULL,
457             document_id   INTEGER,
458             duration_s    INTEGER,
459             width         INTEGER,
460             height        INTEGER,
461             size_bytes    INTEGER,
462             mime_type     TEXT,
463             matched       INTEGER NOT NULL DEFAULT 0,
464             forwarded     INTEGER NOT NULL DEFAULT 0,
465             seen_at       TEXT NOT NULL,
466             UNIQUE (user_id, channel_id, message_id)
467         )

```

```

468         """
469         "id, user_id, channel_id, message_id, document_id, duration_s, width, "
470         "height, size_bytes, mime_type, matched, forwarded, seen_at",
471     ),
472     (
473         "filter_config",
474         """
475         CREATE TABLE filter_config (
476             user_id      INTEGER PRIMARY KEY REFERENCES accounts(id),
477             config_json TEXT NOT NULL
478         )
479         """
480         "user_id, config_json",
481     ),
482 ):
483     await conn.execute(f"ALTER TABLE {table} RENAME TO {table}__legacy_data")
484     await conn.execute(new_ddl)
485     await conn.execute(
486         f"INSERT INTO {table} ({copy_cols}) SELECT {copy_cols} FROM
{table}__legacy_data"
487     )
488     await conn.execute(f"DROP TABLE {table}__legacy_data")
489
490     # The partial unique index for document_id dedup (created by migration 001
491     # against the legacy videos table) was dropped along with the legacy copy
492     # above. Recreate it on the new videos table.
493     await conn.execute(
494         "CREATE UNIQUE INDEX IF NOT EXISTS idx_videos_user_document "
495         "ON videos(user_id, document_id) WHERE document_id IS NOT NULL"
496     )
497
498     # Rename the legacy users table (do NOT drop ? kept as a rollback safety
499     # net for one release per ADR 0010). The data-holding channels/videos/
500     # filter_config tables were already rebuilt above with FKs to accounts(id),
501     # so renaming users no longer retargets any live FK.
502     await conn.execute("ALTER TABLE users RENAME TO users_deprecated")
503
504
505 @_migration(3)
506 async def _migration_003_domain_schema(conn: aiosqlite.Connection) -> None:
507     """Land the provider-neutral domain tables (ADR 0011, Track B).
508
509     Creates ``sources``, ``content_items``, ``policies``, ``destinations``,
510     ``routing_rules``, ``delivery_attempts`` (all ``IF NOT EXISTS`` ? on fresh
511     DBs ``_SCHEMA`` already created them; on pre-ADR-0011 DBs this creates
512     them), then backfills the three populated ones from the legacy
513     ``channels`` / ``videos`` / ``filter_config`` tables and renames the
514     legacy tables to ``*_deprecated`` (one-release rollback net, ADR 0010
515     pattern). Idempotent: a no-op once the legacy tables have been renamed.
516     """
517     # Idempotency gate: if the legacy videos table is gone, this migration has
518     # already run (or the DB is fresh and _SCHEMA's legacy tables were already
519     # dropped by _apply_schema post-003). Nothing to backfill.
520     if not await _table_exists(conn, "videos"):
521         return
522
523     # 1a. Backfill sources from channels first (known title + watched=1). Order
524     # matters: channels-first so title/watched data wins; the videos-based
525     # backfill below fills gaps for orphan channels (review finding #1).
526     await conn.execute(
527         """
528         INSERT OR IGNORE INTO sources
529             (account_id, provider, provider_source_id, title, kind, watched,
530              seen_first_at, last_seen_at)
531         SELECT user_id, 'telegram', CAST(channel_id AS TEXT), title, 'channel',

```

```

532         1, seen_first_at, seen_first_at
533     FROM channels
534     """"
535 )
536
537 # 1b. Backfill sources from videos for channels that have no channels row
538 #     (e.g. backfill-only channels that were never seen via live traffic).
539 #     INSERT OR IGNORE is a no-op on (account, provider, source_id) conflicts
540 #     ? the channels-based row above survives with its title + watched=1.
541 await conn.execute(
542     """"
543     INSERT OR IGNORE INTO sources
544         (account_id, provider, provider_source_id, kind, watched, seen_first_at)
545     SELECT v.user_id, 'telegram', CAST(v.channel_id AS TEXT),
546         'channel', 0, MIN(v.seen_at)
547     FROM videos v
548     GROUP BY v.user_id, v.channel_id
549     """"
550 )
551
552 # 2. Backfill content_items from videos in two phases (review findings #2 & #3):
553 #     Phase A ? insert every row with a guaranteed-unique tg:msg: Tier-0 key
554 #     (no silent drops, even on fingerprint collisions).
555 #     Phase B ? promote to tg:doc: for items with a document_id (one per doc-id
group).
556 #     Phase C ? promote remaining non-doc items to tg:fp: (one per fingerprint
group).
557 #     The non-promoted losers stay at tg:msg: (unique) ? no silent data loss.
558 await conn.execute(
559     """"
560     INSERT INTO content_items
561         (account_id, source_id, provider, provider_message_id, content_type,
562         document_id, duration_s, width, height, size_bytes, mime_type,
563         dedup_key, matched, seen_at)
564     SELECT
565         v.user_id,
566         s.id,
567         'telegram',
568         CAST(v.message_id AS TEXT),
569         'video',
570         CASE WHEN v.document_id IS NOT NULL THEN CAST(v.document_id AS TEXT) END,
571         v.duration_s, v.width, v.height, v.size_bytes, v.mime_type,
572         'tg:msg:' || s.id || ':' || v.message_id, -- Tier-0 (always unique)
573         v.matched,
574         v.seen_at
575     FROM videos v
576     JOIN sources s
577         ON s.account_id = v.user_id
578         AND s.provider = 'telegram'
579         AND s.provider_source_id = CAST(v.channel_id AS TEXT)
580     """"
581 )
582
583 # Phase B: promote one row per (account, document_id) to tg:doc:.
584 await conn.execute(
585     """"
586     UPDATE content_items SET dedup_key = 'tg:doc:' || document_id
587     WHERE document_id IS NOT NULL
588         AND dedup_key LIKE 'tg:msg:%'
589         AND id IN (
590         SELECT MIN(id) FROM content_items
591         WHERE document_id IS NOT NULL AND dedup_key LIKE 'tg:msg:%'
592         GROUP BY account_id, document_id
593         )
594     """"

```

```

595 )
596
597 # Phase C: promote one non-doc row per (account, fingerprint) to tg:fp:.
598 await conn.execute(
599     """
600     UPDATE content_items SET dedup_key =
601         'tg:fp:' || size_bytes || ':' || duration_s || ':'
602         || height || ':' || width
603     WHERE document_id IS NULL
604         AND size_bytes IS NOT NULL AND duration_s IS NOT NULL
605         AND height IS NOT NULL AND width IS NOT NULL
606         AND dedup_key LIKE 'tg:msg:%'
607         AND id IN (
608             SELECT MIN(id) FROM content_items
609             WHERE document_id IS NULL
610                 AND size_bytes IS NOT NULL AND duration_s IS NOT NULL
611                 AND height IS NOT NULL AND width IS NOT NULL
612                 AND dedup_key LIKE 'tg:msg:%'
613             GROUP BY account_id, size_bytes, duration_s, height, width
614         )
615     """
616 )
617
618 # 3. Backfill policies from filter_config: one 'default' policy per account
619 # (Option 2A). is_default=1 so the legacy load_filter_config shim reads
620 # this row. UNIQUE(account_id, name) dedups idempotent re-runs.
621 await conn.execute(
622     """
623     INSERT OR IGNORE INTO policies (account_id, name, config_json, is_default,
624                                     created_at, updated_at)
625     SELECT user_id, 'default', config_json, 1,
626            strftime('%Y-%m-%dT%H:%M:%SZ', 'now'),
627            strftime('%Y-%m-%dT%H:%M:%SZ', 'now')
628     FROM filter_config
629     """
630 )
631
632 # 4. Create the policies default-partial index (sig after backfill so the
633 # UNIQUE constraint only applies once rows are populated). On fresh DBs
634 # _SCHEMA creates it; on migrated DBs this is the first time.
635 await conn.execute(
636     "CREATE UNIQUE INDEX IF NOT EXISTS idx_policies_account_default "
637     "ON policies(account_id) WHERE is_default = 1"
638 )
639
640 # 5. Assert no silent video loss: on a legacy DB, every videos row MUST
641 # produce a content_items row. A mismatch signals fingerprint collisions
642 # where two distinct videos shared (size,dur,h,w) ? the losers stay at
643 # tg:msg: keys and are still present. Log a warning so the operator can
644 # investigate; the data is preserved in *_deprecated tables.
645 async with conn.execute("SELECT COUNT(*) FROM videos") as cur:
646     vc = (await cur.fetchone())[0]
647 async with conn.execute("SELECT COUNT(*) FROM content_items") as cur:
648     cic = (await cur.fetchone())[0]
649 if vc != cic:
650     import logging
651
652     _logger = logging.getLogger(__name__)
653     _logger.warning(
654         "migration_003: %d videos rows produced %d content_items rows "
655         "%d gap). Colliding items remain in videos_deprecated.",
656         vc,
657         cic,
658         vc - cic,
659     )

```

```

660
661 # 6. Rename (don't drop) the legacy tables ? kept for one release as a
662 # rollback safety net (ADR 0010 pattern).
663 await conn.execute("ALTER TABLE channels RENAME TO channels_deprecated")
664 await conn.execute("ALTER TABLE videos RENAME TO videos_deprecated")
665 await conn.execute("ALTER TABLE filter_config RENAME TO filter_config_deprecated")
666
667
668 @_migration(4)
669 async def _migration_004_destinations_and_routing(conn: aiosqlite.Connection) -> None:
670     """Backfill default destinations and delivery state (ADR 0011, Track C).
671
672     Creates a default destination row for each account that has a
673     ``filter_config_deprecated.delivery_target`` value, then creates a default
674     routing rule (source ? default policy ? destination) for each source.
675     Also backfills ``delivery_attempts`` rows from the legacy ``videos_deprecated``
676     table (one ``delivered`` attempt per previously-forwarded video to the
677     account's default destination).
678
679     Idempotent: skips accounts whose default destination already exists.
680     """
681     # Gate: if destinations already have data, this migration has run.
682     async with conn.execute("SELECT COUNT(*) FROM destinations") as cur:
683         if (await cur.fetchone())[0] > 0:
684             return
685
686     if not await _table_exists(conn, "filter_config_deprecated"):
687         return
688
689     # 1. Backfill default destinations from filter_config_deprecated.
690     # Parse the JSON delivery_target, defaulting to "me".
691     await conn.execute(
692         """
693         INSERT OR IGNORE INTO destinations (account_id, provider, target, kind,
694         is_output, created_at)
695         SELECT user_id, 'telegram',
696             COALESCE(
697                 CASE WHEN json_valid(config_json)
698                     THEN json_extract(config_json, '$.delivery_target') END,
699                 'me'
700             ),
701             CASE
702                 WHEN COALESCE(
703                     CASE WHEN json_valid(config_json)
704                         THEN json_extract(config_json, '$.delivery_target') END,
705                     'me'
706                 ) = 'me'
707                 THEN 'saved'
708                 ELSE 'channel'
709             END,
710             0,
711             strftime('%Y-%m-%dT%H:%M:%SZ', 'now')
712         FROM filter_config_deprecated
713         """
714     )
715
716     # 2. Create a default routing rule per source (source ? default policy ? default
717     # destination).
718     await conn.execute(
719         """
720         INSERT OR IGNORE INTO routing_rules
721         (account_id, source_id, policy_id, destination_id, enabled, priority,
722         created_at)
723         SELECT s.account_id, s.id, p.id, d.id, 1, 0,
724             strftime('%Y-%m-%dT%H:%M:%SZ', 'now')

```

```

722         FROM sources s
723         JOIN policies p ON p.account_id = s.account_id AND p.is_default = 1
724         JOIN destinations d ON d.account_id = s.account_id
725         WHERE d.target = (
726             SELECT COALESCE(
727                 CASE WHEN json_valid(fcd.config_json)
728                     THEN json_extract(fcd.config_json, '$.delivery_target') END,
729                 'me'
730             )
731         FROM filter_config_deprecated fcd
732         WHERE fcd.user_id = s.account_id
733         LIMIT 1
734     )
735     """
736 )
737
738 # 3. Backfill delivery_attempts from previously-forwarded videos.
739 await conn.execute(
740     """
741     INSERT OR IGNORE INTO delivery_attempts
742         (content_item_id, destination_id, route_id, status, attempt_count,
743          created_at, delivered_at)
744     SELECT ci.id, d.id, rr.id, 'delivered', 1,
745            vd.seen_at, vd.seen_at
746     FROM videos_deprecated vd
747     JOIN content_items ci
748         ON ci.account_id = vd.user_id
749         AND ci.provider = 'telegram'
750         AND ci.provider_message_id = CAST(vd.message_id AS TEXT)
751     JOIN sources sc
752         ON sc.id = ci.source_id
753         AND sc.provider_source_id = CAST(vd.channel_id AS TEXT)
754     JOIN destinations d ON d.account_id = vd.user_id
755     JOIN routing_rules rr
756         ON rr.account_id = vd.user_id
757         AND rr.source_id = sc.id
758         AND rr.destination_id = d.id
759     WHERE vd.forwarded = 1
760     """
761 )
762
763
764 async def _table_exists(conn: aiosqlite.Connection, table: str) -> bool:
765     """Return True if ``table`` exists (async, PRAGMA table_list / sqlite_master)."""
766     async with conn.execute(
767         "SELECT 1 FROM sqlite_master WHERE type='table' AND name=?", (table,)
768     ) as cur:
769         return await cur.fetchone() is not None
770
771
772 # --- helpers -----
773
774
775 def _now() -> datetime:
776     """Timezone-aware UTC 'now', used for policy timestamps."""
777     return datetime.now(UTC)
778
779
780 def _safe_int_cast(value: object, field_name: str) -> int:
781     """Cast a provider_source_id / provider_message_id / document_id to int.
782
783     All existing data is Telegram ? these ids are numeric strings. Non-Telegram
784     providers cannot be projected onto the legacy ``Channel`` / ``VideoRecord``
785     shapes (which carry Telegram-specific int ids); this function raises a clear
786     ``ValueError`` rather than letting a cryptic ``int()`` crash propagate

```

```

787 (ADR 0011 provider-neutrality guard ? review finding #5).
788 """
789 try:
790     return int(value) # type: ignore[arg-type]
791 except (ValueError, TypeError) as exc:
792     raise ValueError(
793         f"Cannot cast {field_name}={value!r} to int. The legacy Channel/VideoRecord
794         f"types are Telegram-specific; use ContentItem/Source directly for "
795         f"non-Telegram providers (ADR 0011).\"
796     ) from exc
797
798
799 def _row_to_account(row: aiosqlite.Row) -> Account:
800     return Account(
801         id=row["id"],
802         telegram_id=row["telegram_id"],
803         provider=row["provider"],
804         display_name=row["display_name"],
805         created_at=row["created_at"], # stored as ISO string; pydantic parses
806     )
807
808
809 def _row_to_telegram_session(row: aiosqlite.Row) -> TelegramSession:
810     return TelegramSession(
811         account_id=row["account_id"],
812         phone=row["phone"],
813         session_path=row["session_path"],
814         enabled=bool(row["enabled"]),
815         created_at=row["created_at"],
816         last_login_at=row["last_login_at"],
817     )
818
819
820 def _account_session_to_user(account: Account, session: TelegramSession | None) -> User:
821     """Project an (account, optional session) pair onto the legacy ``User`` view.
822
823     Used by the deprecated ``upsert_user``/``get_user``/``list_enabled_users``
824     shims so pre-ADR-0010 callers and tests keep working. A bot-only account
825     (no session) maps to a ``User`` with ``enabled=False`` so the legacy
826     "list enabled users" semantics exclude it ? matching the structural rule
827     that bot-only users are not runnable Telethon clients.
828     """
829     if session is None:
830         return User(
831             id=account.telegram_id if account.telegram_id is not None else account.id,
832             phone=None,
833             enabled=False,
834             created_at=account.created_at,
835         )
836     return User(
837         id=account.telegram_id if account.telegram_id is not None else account.id,
838         phone=session.phone,
839         enabled=session.enabled,
840         created_at=session.created_at,
841     )
842
843
844 def _row_to_source(row: aiosqlite.Row) -> Source:
845     return Source(
846         id=row["id"],
847         account_id=row["account_id"],
848         provider=row["provider"],
849         provider_source_id=row["provider_source_id"],
850         title=row["title"],

```



```

851         kind=row["kind"],
852         watched=bool(row["watched"]),
853         seen_first_at=row["seen_first_at"],
854         last_seen_at=row["last_seen_at"],
855     )
856
857
858 def _row_to_content_item(row: aiosqlite.Row) -> ContentItem:
859     return ContentItem(
860         id=row["id"],
861         account_id=row["account_id"],
862         source_id=row["source_id"],
863         provider=row["provider"],
864         provider_message_id=row["provider_message_id"],
865         content_type=row["content_type"],
866         text=row["text"],
867         link_url=row["link_url"],
868         media_ref=row["media_ref"],
869         document_id=row["document_id"],
870         duration_s=row["duration_s"],
871         width=row["width"],
872         height=row["height"],
873         size_bytes=row["size_bytes"],
874         mime_type=row["mime_type"],
875         dedup_key=row["dedup_key"],
876         matched=bool(row["matched"]),
877         seen_at=row["seen_at"],
878     )
879
880
881 def _row_to_policy(row: aiosqlite.Row) -> Policy:
882     return Policy(
883         id=row["id"],
884         account_id=row["account_id"],
885         name=row["name"],
886         config_json=row["config_json"],
887         is_default=bool(row["is_default"]),
888         created_at=row["created_at"],
889         updated_at=row["updated_at"],
890     )
891
892
893 def _row_to_destination(row: aiosqlite.Row) -> Destination:
894     return Destination(
895         id=row["id"],
896         account_id=row["account_id"],
897         provider=row["provider"],
898         target=row["target"],
899         kind=row["kind"],
900         title=row["title"],
901         is_output=bool(row["is_output"]),
902         created_at=row["created_at"],
903     )
904
905
906 def _row_to_routing_rule(row: aiosqlite.Row) -> RoutingRule:
907     return RoutingRule(
908         id=row["id"],
909         account_id=row["account_id"],
910         source_id=row["source_id"],
911         policy_id=row["policy_id"],
912         destination_id=row["destination_id"],
913         enabled=bool(row["enabled"]),
914         priority=row["priority"],
915         created_at=row["created_at"],

```

```

916     )
917
918
919 def _row_to_delivery_attempt(row: aiosqlite.Row) -> DeliveryAttempt:
920     return DeliveryAttempt(
921         id=row["id"],
922         content_item_id=row["content_item_id"],
923         destination_id=row["destination_id"],
924         route_id=row["route_id"],
925         status=DeliveryStatus(row["status"]),
926         attempt_count=row["attempt_count"],
927         last_error=row["last_error"],
928         created_at=row["created_at"],
929         delivered_at=row["delivered_at"],
930         next_retry_at=row["next_retry_at"],
931     )
932
933
934 # --- account + session repos (ADR 0010) -----
935
936
937 async def upsert_account(conn: aiosqlite.Connection, account: Account) -> Account:
938     """Insert or update an account row; returns the persisted Account."""
939     await conn.execute(
940         """INSERT INTO accounts (id, telegram_id, provider, display_name, created_at)
941         VALUES (?, ?, ?, ?, ?)
942         ON CONFLICT(id) DO UPDATE SET
943             telegram_id=excluded.telegram_id,
944             provider=excluded.provider,
945             display_name=excluded.display_name""",
946         (
947             account.id,
948             account.telegram_id,
949             account.provider,
950             account.display_name,
951             account.created_at.isoformat(),
952         ),
953     )
954     await conn.commit()
955     return await get_account(conn, account.id) # type: ignore[return-value]
956
957
958 async def get_account(conn: aiosqlite.Connection, account_id: int) -> Account | None:
959     conn.row_factory = aiosqlite.Row
960     async with conn.execute("SELECT * FROM accounts WHERE id = ?", (account_id,)) as
cur:
961         row = await cur.fetchone()
962         return _row_to_account(row) if row else None
963
964
965 async def get_account_by_telegram_id(
966     conn: aiosqlite.Connection, telegram_id: int
967 ) -> Account | None:
968     """Resolve a Bot API ``from.id`` (or userbot account id) to an Account."""
969     conn.row_factory = aiosqlite.Row
970     async with conn.execute("SELECT * FROM accounts WHERE telegram_id = ?",
(telegram_id,)) as cur:
971         row = await cur.fetchone()
972         return _row_to_account(row) if row else None
973
974
975 async def upsert_telegram_session(
976     conn: aiosqlite.Connection, session: TelegramSession
977 ) -> TelegramSession:
978     """Insert or update a telegram_sessions row; returns the persisted session."""

```

```

979     await conn.execute(
980         """INSERT INTO telegram_sessions
981             (account_id, phone, session_path, enabled, created_at, last_login_at)
982             VALUES (?, ?, ?, ?, ?, ?)
983             ON CONFLICT(account_id) DO UPDATE SET
984                 phone=excluded.phone,
985                 session_path=excluded.session_path,
986                 enabled=excluded.enabled,
987                 last_login_at=excluded.last_login_at""",
988         (
989             session.account_id,
990             session.phone,
991             session.session_path,
992             int(session.enabled),
993             session.created_at.isoformat(),
994             session.last_login_at.isoformat() if session.last_login_at else None,
995         ),
996     )
997     await conn.commit()
998     conn.row_factory = aiosqlite.Row
999     async with conn.execute(
1000         "SELECT * FROM telegram_sessions WHERE account_id = ?", (session.account_id,)
1001     ) as cur:
1002         row = await cur.fetchone()
1003         # The row must exist ? we just upserted it by PK (account_id). Assert the
1004         # invariant explicitly so mypy can narrow Row | None ? Row without a
1005         # type-ignore comment masking a real future regression.
1006         assert row is not None
1007         return _row_to_telegram_session(row)
1008
1009
1010     async def get_telegram_session(
1011         conn: aiosqlite.Connection, account_id: int
1012     ) -> TelegramSession | None:
1013         conn.row_factory = aiosqlite.Row
1014         async with conn.execute(
1015             "SELECT * FROM telegram_sessions WHERE account_id = ?", (account_id,)
1016         ) as cur:
1017             row = await cur.fetchone()
1018             return _row_to_telegram_session(row) if row else None
1019
1020
1021     async def list_runnable_sessions(
1022         conn: aiosqlite.Connection,
1023     ) -> list[tuple[Account, TelegramSession]]:
1024         """Return (account, session) pairs for sessions the manager should start.
1025
1026         This is the ADR 0010 structural rule: ``manager.run`` iterates this list,
1027         not ``accounts``. A bot-only user has no ``telegram_sessions`` row and is
1028         therefore inherently invisible to the Telethon supervisor ? the "don't
1029         start a client for bot-only users" invariant is enforced by query shape.
1030         """
1031         conn.row_factory = aiosqlite.Row
1032         async with conn.execute(
1033             """SELECT a.*, s.phone AS s_phone, s.session_path, s.enabled AS s_enabled,
1034                 s.created_at AS s_created_at, s.last_login_at
1035             FROM accounts a
1036             JOIN telegram_sessions s ON s.account_id = a.id
1037             WHERE s.enabled = 1"""
1038         ) as cur:
1039             rows = await cur.fetchall()
1040             return [
1041                 (
1042                     _row_to_account(r),
1043                     TelegramSession(

```

```

1044         account_id=r["id"],
1045         phone=r["s_phone"],
1046         session_path=r["session_path"],
1047         enabled=bool(r["s_enabled"]),
1048         created_at=r["s_created_at"],
1049         last_login_at=r["last_login_at"],
1050     ),
1051 )
1052 for r in rows
1053 ]
1054
1055
1056 # --- legacy User shims (backwards-compatible view over accounts + sessions) --
1057
1058
1059 async def upsert_user(conn: aioredis.Connection, user: User) -> User:
1060     """Deprecated shim: upsert the (account, session) pair implied by a ``User``.
1061
1062     Maps the legacy v1 ``User`` onto the ADR 0010 schema: ``User.id`` becomes
1063     both ``accounts.id`` and ``accounts.telegram_id`` (legacy users are
1064     Telegram userbot accounts), and a ``telegram_sessions`` row is created so
1065     the manager can run them. ``User.enabled`` maps to ``session.enabled``.
1066     """
1067     account = Account(id=user.id, telegram_id=user.id, created_at=user.created_at)
1068     await upsert_account(conn, account)
1069     await upsert_telegram_session(
1070         conn,
1071         TelegramSession(
1072             account_id=user.id,
1073             phone=user.phone,
1074             session_path=f"user_{user.id}.session",
1075             enabled=user.enabled,
1076             created_at=user.created_at,
1077         ),
1078     )
1079     return await get_user(conn, user.id) # type: ignore[return-value]
1080
1081
1082 async def get_user(conn: aioredis.Connection, user_id: int) -> User | None:
1083     """Deprecated shim: project the (account, session) pair back to a ``User``. """
1084     account = await get_account(conn, user_id)
1085     if account is None:
1086         return None
1087     session = await get_telegram_session(conn, user_id)
1088     return _account_session_to_user(account, session)
1089
1090
1091 async def list_enabled_users(conn: aioredis.Connection) -> list[User]:
1092     """Deprecated shim: list users that have an enabled MTPROTO session.
1093
1094     Equivalent to projecting :func:`list_runnable_sessions` onto the legacy
1095     ``User`` shape. Bot-only accounts (no session) are excluded ? matching the
1096     pre-ADR-0010 "list enabled users" semantics where every row was runnable.
1097     """
1098     pairs = await list_runnable_sessions(conn)
1099     return [_account_session_to_user(a, s) for a, s in pairs]
1100
1101
1102 # --- Source repos (ADR 0011) -----
1103
1104
1105 async def upsert_source(conn: aioredis.Connection, source: Source) -> Source:
1106     """Insert a source row, or refresh title/last_seen_at on conflict.
1107
1108     The conflict key is ``(account_id, provider, provider_source_id)`` per the

```

```

1109     ADR 0011 schema. Returns the persisted row.
1110
1111     ``watched`` is **user-controlled state** (toggled only via
1112     :func:`set_source_watched` / the ``/watch`` and ``/unwatch`` commands) and is
1113     therefore **never overwritten on conflict** ? the ingestion path
1114     (``upsert_channel`` / ``insert_video``) re-touches a source on every message,
1115     so clobbering ``watched`` here would silently un-watch a channel the user
1116     opted into after its first ingested video (ADR 0009). ``watched`` is only set
1117     on the initial INSERT of a new row. ``title`` is refreshed but never nulled:
1118     the ingestion shims pass ``title=None``, so ``COALESCE`` keeps the existing
1119     title rather than erasing the channel name.
1120     """
1121     await conn.execute(
1122         """INSERT INTO sources
1123             (account_id, provider, provider_source_id, title, kind, watched,
1124              seen_first_at, last_seen_at)
1125             VALUES (?, ?, ?, ?, ?, ?, ?, ?)
1126             ON CONFLICT(account_id, provider, provider_source_id) DO UPDATE SET
1127                 title=COALESCE(excluded.title, sources.title),
1128                 last_seen_at=excluded.last_seen_at""",
1129         (
1130             source.account_id,
1131             source.provider,
1132             source.provider_source_id,
1133             source.title,
1134             source.kind,
1135             int(source.watched),
1136             source.seen_first_at.isoformat(),
1137             (source.last_seen_at.isoformat() if source.last_seen_at else None),
1138         ),
1139     )
1140     await conn.commit()
1141     conn.row_factory = aiosqlite.Row
1142     async with conn.execute(
1143         """SELECT * FROM sources
1144             WHERE account_id = ? AND provider = ? AND provider_source_id = ?""",
1145         (source.account_id, source.provider, source.provider_source_id),
1146     ) as cur:
1147         row = await cur.fetchone()
1148         assert row is not None # just upserted by the conflict key
1149         return _row_to_source(row)
1150
1151
1152     async def get_source_by_provider_id(
1153         conn: aiosqlite.Connection, account_id: int, provider: str, provider_source_id: str
1154     ) -> Source | None:
1155         """Resolve a source by its provider-native id."""
1156         conn.row_factory = aiosqlite.Row
1157         async with conn.execute(
1158             """SELECT * FROM sources
1159                 WHERE account_id = ? AND provider = ? AND provider_source_id = ?""",
1160             (account_id, provider, provider_source_id),
1161         ) as cur:
1162             row = await cur.fetchone()
1163             return _row_to_source(row) if row else None
1164
1165
1166     # --- ContentItem repos (ADR 0011) -----
1167
1168
1169     async def insert_content_item(conn: aiosqlite.Connection, item: ContentItem) ->
ContentItem:
1170         """Insert a content item. Raises ``IntegrityError`` on a dedup collision.
1171
1172         The ``dedup_key`` must be precomputed by :func:`compute_dedup_key`. Two

```

```

1173 unique constraints guard against duplicates:
1174 - ``UNIQUE (account_id, source_id, provider_message_id)`` ? Tier-0.
1175 - ``UNIQUE (account_id, dedup_key)`` ? Tier-1/2 (provider-neutral).
1176 """
1177 cur = await conn.execute(
1178     """INSERT INTO content_items
1179         (account_id, source_id, provider, provider_message_id, content_type,
1180          text, link_url, media_ref, document_id, duration_s, width, height,
1181          size_bytes, mime_type, dedup_key, matched, seen_at)
1182         VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)""",
1183     (
1184         item.account_id,
1185         item.source_id,
1186         item.provider,
1187         item.provider_message_id,
1188         item.content_type,
1189         item.text,
1190         item.link_url,
1191         item.media_ref,
1192         item.document_id,
1193         item.duration_s,
1194         item.width,
1195         item.height,
1196         item.size_bytes,
1197         item.mime_type,
1198         item.dedup_key,
1199         int(item.matched),
1200         item.seen_at.isoformat(),
1201     ),
1202 )
1203 await conn.commit()
1204 return item.model_copy(update={"id": cur.lastrowid})
1205
1206
1207 async def get_content_item_by_message(
1208     conn: aiosqlite.Connection,
1209     account_id: int,
1210     source_id: int,
1211     provider_message_id: str,
1212 ) -> ContentItem | None:
1213     """Tier-0 lookup by the (account, source, provider-message-id) natural key."""
1214     conn.row_factory = aiosqlite.Row
1215     async with conn.execute(
1216         """SELECT * FROM content_items
1217             WHERE account_id = ? AND source_id = ? AND provider_message_id = ?""",
1218         (account_id, source_id, provider_message_id),
1219     ) as cur:
1220         row = await cur.fetchone()
1221         return _row_to_content_item(row) if row else None
1222
1223
1224 async def get_content_item_by_dedup_key(
1225     conn: aiosqlite.Connection, account_id: int, dedup_key: str
1226 ) -> ContentItem | None:
1227     """Provider-neutral dedup lookup by ``dedup_key`` (Option 1A).
1228
1229     Replaces the Tier-1 (``document_id``) and Tier-2 (fingerprint) lookups with
1230     a single query: the ``dedup_key`` already encodes whichever tier was
1231     applicable at ingestion time.
1232     """
1233     conn.row_factory = aiosqlite.Row
1234     async with conn.execute(
1235         "SELECT * FROM content_items WHERE account_id = ? AND dedup_key = ?",
1236         (account_id, dedup_key),
1237     ) as cur:

```

```

1238         row = await cur.fetchone()
1239         return _row_to_content_item(row) if row else None
1240
1241
1242 # --- Policy repos (ADR 0011, Option 2A) -----
1243
1244
1245 async def upsert_policy(conn: aiomysqlite.Connection, policy: Policy) -> Policy:
1246     """Insert or update a policy row by `(account_id, name)`".
1247
1248     Exactly one policy per account should carry `is_default=True`; callers
1249     are responsible for flipping the previous default before flipping a new
1250     one (enforced by a partial unique index landed by future work, not in the
1251     schema ? SQLite `CHECK` can't express "at most one per group").
1252     """
1253     await conn.execute(
1254         """INSERT INTO policies (account_id, name, config_json, is_default,
1255                                created_at, updated_at)
1256         VALUES (?, ?, ?, ?, ?, ?)
1257         ON CONFLICT(account_id, name) DO UPDATE SET
1258             config_json=excluded.config_json,
1259             is_default=excluded.is_default,
1260             updated_at=excluded.updated_at""",
1261         (
1262             policy.account_id,
1263             policy.name,
1264             policy.config_json,
1265             int(policy.is_default),
1266             policy.created_at.isoformat(),
1267             policy.updated_at.isoformat(),
1268         ),
1269     )
1270     await conn.commit()
1271     conn.row_factory = aiomysqlite.Row
1272     async with conn.execute(
1273         "SELECT * FROM policies WHERE account_id = ? AND name = ?",
1274         (policy.account_id, policy.name),
1275     ) as cur:
1276         row = await cur.fetchone()
1277         assert row is not None
1278         return _row_to_policy(row)
1279
1280
1281 async def get_default_policy(conn: aiomysqlite.Connection, account_id: int) -> Policy |
None:
1282     """Return the account's default policy (`is_default=1`)."""
1283     conn.row_factory = aiomysqlite.Row
1284     async with conn.execute(
1285         "SELECT * FROM policies WHERE account_id = ? AND is_default = 1",
1286         (account_id,),
1287     ) as cur:
1288         row = await cur.fetchone()
1289         return _row_to_policy(row) if row else None
1290
1291
1292 def _policy_config_json_to_dict(raw: str) -> dict[str, object]:
1293     """Decode a policy config JSON object.
1294
1295     ADR 0012 keeps future policy keys in `policies.config_json`. The value is
1296     expected to be a JSON object so legacy flat `FilterConfig` keys and future
1297     nested stage config can coexist.
1298     """
1299     decoded = json.loads(raw)
1300     if not isinstance(decoded, dict):
1301         raise ValueError("policy config_json must decode to a JSON object")

```

```

1302         return decoded
1303
1304
1305     def _policy_config_dict_to_json(config: dict[str, object]) -> str:
1306         """Encode policy config deterministically and compactly."""
1307         return json.dumps(config, separators=(",", ":"), sort_keys=True)
1308
1309
1310     async def load_policy_config(conn: aiolsqlite.Connection, account_id: int) -> dict[str,
object]:
1311         """Load the raw default policy config as a JSON object.
1312
1313         Unlike :func:`load_filter_config`, this preserves future ADR 0012 keys such
1314         as ``topics`` / ``security_checks`` / ``deep_inspect`` instead of validating
1315         through the flat ``FilterConfig`` model.
1316         """
1317         policy = await get_default_policy(conn, account_id)
1318         if policy is None:
1319             return FilterConfig().model_dump(mode="json")
1320         return _policy_config_json_to_dict(policy.config_json)
1321
1322
1323     async def save_policy_config(
1324         conn: aiolsqlite.Connection, account_id: int, config: dict[str, object]
1325     ) -> None:
1326         """Persist the raw default policy config object."""
1327         existing = await get_default_policy(conn, account_id)
1328         await upsert_policy(
1329             conn,
1330             Policy(
1331                 id=existing.id if existing is not None else None,
1332                 account_id=account_id,
1333                 name="default",
1334                 config_json=_policy_config_dict_to_json(config),
1335                 is_default=True,
1336                 created_at=(existing.created_at if existing is not None else _now()),
1337                 updated_at=_now(),
1338             ),
1339         )
1340
1341
1342     # --- Destination / RoutingRule / DeliveryAttempt repos (ADR 0011, Track C) --
1343
1344
1345     async def upsert_destination(conn: aiolsqlite.Connection, dest: Destination) ->
Destination:
1346         """Insert or update a destination row by (account, provider, target)."""
1347         await conn.execute(
1348             """INSERT INTO destinations
1349                 (account_id, provider, target, kind, title, is_output, created_at)
1350                 VALUES (?, ?, ?, ?, ?, ?, ?)
1351                 ON CONFLICT(account_id, provider, target) DO UPDATE SET
1352                     kind=excluded.kind,
1353                     title=excluded.title,
1354                     is_output=excluded.is_output""",
1355             (
1356                 dest.account_id,
1357                 dest.provider,
1358                 dest.target,
1359                 dest.kind,
1360                 dest.title,
1361                 int(dest.is_output),
1362                 dest.created_at.isoformat(),
1363             ),
1364         )

```



```

1365         await conn.commit()
1366         conn.row_factory = aiosqlite.Row
1367         async with conn.execute(
1368             """SELECT * FROM destinations
1369                 WHERE account_id = ? AND provider = ? AND target = ?""",
1370             (dest.account_id, dest.provider, dest.target),
1371         ) as cur:
1372             row = await cur.fetchone()
1373         assert row is not None
1374         return _row_to_destination(row)
1375
1376
1377     async def get_destination(
1378         conn: aiosqlite.Connection, account_id: int, provider: str, target: str
1379     ) -> Destination | None:
1380         """Resolve a destination by its (account, provider, target) natural key."""
1381         conn.row_factory = aiosqlite.Row
1382         async with conn.execute(
1383             "SELECT * FROM destinations WHERE account_id = ? AND provider = ? AND target = ?",
1384             (account_id, provider, target),
1385         ) as cur:
1386             row = await cur.fetchone()
1387         return _row_to_destination(row) if row else None
1388
1389

```

5. assistant (2026-07-06T20:52:46.389Z)

Now let me look at the git diff to see exactly what changed, and check the models and tests.

6. user (2026-07-06T20:52:49.610Z)

```

diff --git a/src/tg_video_filter/db.py b/src/tg_video_filter/db.py
index a059e57..7f7e400 100644
--- a/src/tg_video_filter/db.py
+++ b/src/tg_video_filter/db.py
@@ -11,6 +11,7 @@ in-memory database (`aiosqlite.connect(":memory:")`).

 from __future__ import annotations

+import json
+from collections.abc import Awaitable, Callable, Iterable
+from datetime import UTC, datetime

@@ -1288,6 +1289,56 @@ async def get_default_policy(conn: aiosqlite.Connection, account_id: int)
-> Pol
    return _row_to_policy(row) if row else None

+def _policy_config_json_to_dict(raw: str) -> dict[str, object]:
+    """Decode a policy config JSON object.
+
+    ADR 0012 keeps future policy keys in ``policies.config_json``. The value is
+    expected to be a JSON object so legacy flat ``FilterConfig`` keys and future
+    nested stage config can coexist.
+    """
+    decoded = json.loads(raw)
+    if not isinstance(decoded, dict):
+        raise ValueError("policy config_json must decode to a JSON object")
+    return decoded
+
+def _policy_config_dict_to_json(config: dict[str, object]) -> str:
+    """Encode policy config deterministically and compactly."""

```

```

+     return json.dumps(config, separators=(",", ":"), sort_keys=True)
+
+
+async def load_policy_config(conn: aioredis.Connection, account_id: int) -> dict[str, object]:
+    """Load the raw default policy config as a JSON object.
+
+    Unlike :func:`load_filter_config`, this preserves future ADR 0012 keys such
+    as ``topics`` / ``security_checks`` / ``deep_inspect`` instead of validating
+    through the flat ``FilterConfig`` model.
+    """
+    policy = await get_default_policy(conn, account_id)
+    if policy is None:
+        return FilterConfig().model_dump(mode="json")
+    return _policy_config_json_to_dict(policy.config_json)
+
+
+async def save_policy_config(
+    conn: aioredis.Connection, account_id: int, config: dict[str, object]
+) -> None:
+    """Persist the raw default policy config object."""
+    existing = await get_default_policy(conn, account_id)
+    await upsert_policy(
+        conn,
+        Policy(
+            id=existing.id if existing is not None else None,
+            account_id=account_id,
+            name="default",
+            config_json=_policy_config_dict_to_json(config),
+            is_default=True,
+            created_at=(existing.created_at if existing is not None else _now()),
+            updated_at=_now(),
+        ),
+    )
+
+
+    # --- Destination / RoutingRule / DeliveryAttempt repos (ADR 0011, Track C) --

```

```

@@ -1705,24 +1756,26 @@ async def load_filter_config(conn: aioredis.Connection, user_id: int)
-> Filter
    policy = await get_default_policy(conn, user_id)
    if policy is None:
        return FilterConfig()
-    return FilterConfig.model_validate_json(policy.config_json)
+    return FilterConfig.model_validate(_policy_config_json_to_dict(policy.config_json))

```

```

    async def save_filter_config(conn: aioredis.Connection, user_id: int, cfg: FilterConfig) ->
None:
-    """Deprecated shim: upsert the account's default :class:`Policy`."""
+    """Deprecated shim: update flat ``FilterConfig`` keys on the default policy.
+
+    ADR 0012: future pipeline keys live in the same ``policies.config_json``
+    object. Do not overwrite the whole blob with ``cfg.model_dump_json()``;
+    merge the flat config fields into the existing object so unknown policy
+    keys survive legacy ``/set`` command updates.
+    """
    existing = await get_default_policy(conn, user_id)
-    await upsert_policy(
-        conn,
-        Policy(
-            id=existing.id if existing is not None else None,
-            account_id=user_id,
-            name="default",
-            config_json=cfg.model_dump_json(),

```

```

-         is_default=True,
-         created_at=(existing.created_at if existing is not None else _now()),
-         updated_at=_now(),
-     ),
- )
+ raw_config: dict[str, object] = {}
+ if existing is not None:
+     try:
+         raw_config = _policy_config_json_to_dict(existing.config_json)
+     except (json.JSONDecodeError, ValueError):
+         raw_config = {}
+ raw_config.update(cfg.model_dump(mode="json"))
+ await save_policy_config(conn, user_id, raw_config)

```

```

# --- bulk helpers (used by the manager on startup) -----
@@ -1789,6 +1842,8 @@ __all__ = [
    "get_content_item_by_dedup_key",
    "upsert_policy",
    "get_default_policy",
+   "load_policy_config",
+   "save_policy_config",
    # Destination / Routing / Delivery repos (ADR 0011, Track C)
    "upsert_destination",
    "get_destination",

```

7. user (2026-07-06T20:52:50.388Z)

2aad4d6 docs(adr): propose 0012 policy pipeline ? explainable, tiered, user-customizable content gates (Track F) (#40)

793a8db feat(policy): add F1 pipeline primitives

2aad4d6 docs(adr): propose 0012 policy pipeline ? explainable, tiered, user-customizable content gates (Track F) (#40)

c4c4d18 fix(scanning): don't clobber user-controlled watched on source upsert (#41)

3bfb1da feat(scanning): selective source scanning ? /watch, /unwatch, /channels, /watchlist (ADR 0009, Track E) (#39)

35f4f70 feat(commands): extract CommandService ? provider-independent command execution (ADR 0011, Track D) (#38)

8. assistant (2026-07-06T20:52:52.756Z)

Now let me look at the full `load\_filter\_config`/`save\_filter\_config` context and the FilterConfig model to understand the merge semantics.

9. user (2026-07-06T20:52:54.407Z)

```

1745
1746 # --- filter config shims (backwards-compatible view over policies) -----
1747
1748
1749 async def load_filter_config(conn: aiomysqlite.Connection, user_id: int) -> FilterConfig:
1750     """Deprecated shim: load the account's default :class:`Policy` as FilterConfig.
1751
1752     ADR 0011 (Option 2A): ``filter_config`` is renamed to
1753     ``filter_config_deprecated``; this reads ``policies WHERE is_default=1``
1754     instead. Falls back to :data:`FilterConfig` defaults if no policy exists.
1755     """
1756     policy = await get_default_policy(conn, user_id)
1757     if policy is None:
1758         return FilterConfig()
1759     return FilterConfig.model_validate(_policy_config_json_to_dict(policy.config_json))
1760
1761

```

```

1762     async def save_filter_config(conn: aiomysqlite.Connection, user_id: int, cfg:
FilterConfig) -> None:
1763         """Deprecated shim: update flat ``FilterConfig`` keys on the default policy.
1764
1765         ADR 0012: future pipeline keys live in the same ``policies.config_json``
1766         object. Do not overwrite the whole blob with ``cfg.model_dump_json()``;
1767         merge the flat config fields into the existing object so unknown policy
1768         keys survive legacy ``/set`` command updates.
1769         """
1770         existing = await get_default_policy(conn, user_id)
1771         raw_config: dict[str, object] = {}
1772         if existing is not None:
1773             try:
1774                 raw_config = _policy_config_json_to_dict(existing.config_json)
1775             except (json.JSONDecodeError, ValueError):
1776                 raw_config = {}
1777             raw_config.update(cfg.model_dump(mode="json"))
1778             await save_policy_config(conn, user_id, raw_config)
1779
1780
1781 # --- bulk helpers (used by the manager on startup) -----
1782
1783
1784     async def list_videos_for_user( # v2: used by on-demand /search; not called in v1
1785         conn: aiomysqlite.Connection, user_id: int, limit: int = 100
1786     ) -> list[VideoRecord]:
1787         """Deprecated shim: list the account's content items projected onto VideoRecord."""
1788         conn.row_factory = aiomysqlite.Row
1789         async with conn.execute(
1790             """SELECT ci.*, s.provider_source_id AS src_pid
1791                FROM content_items ci
1792                JOIN sources s ON s.id = ci.source_id
1793                WHERE ci.account_id = ?
1794                ORDER BY ci.id DESC LIMIT ?""",

```

10. user (2026-07-06T20:52:55.152Z)

```

299:class FilterConfig(BaseModel):
300-    """Per-user filter configuration.
301-
302-    v1 rules: duration bounds, minimum height (resolution), and max file size.
303-    Stored in the DB as a JSON blob (``filter_config.config_json``) so adding a
304-    new field later is backwards-compatible.
305-    """
306-
307-    # Duration bounds in seconds. None means unbounded on that side.
308-    min_duration_s: int | None = Field(default=5, ge=0)
309-    max_duration_s: int | None = Field(default=7200, ge=0)
310-
311-    # Resolution: a video passes if its height >= target_height.
312-    # None disables the resolution rule.
313-    target_height: int | None = Field(default=1080, ge=1)
314-
315-    # Max file size in mebibytes. None disables the size rule.
316-    max_size_mb: int | None = Field(default=2048, ge=1)
317-
318-    # Content-type exclusions. GIFs and round videos are excluded by default
319-    # (their duration/dimensions are usually incidental to real video filters);
320-    # silent videos are off by default since some legit clips have no audio.
321-    # Existing users' stored configs are unaffected ? the DB JSON blob overrides
322-    # these defaults, so this only changes the behaviour for fresh installs.
323-    exclude_gif: bool = Field(
324-        default=True,
325-        description="If True, exclude Telegram GIFs (DocumentAttributeAnimated)",
326-    )

```

```

327-     exclude_round: bool = Field(
328-         default=True,
329-         description="If True, exclude circular video messages (round_message=True)",
330-     )
331-     exclude_nosound: bool = Field(
332-         default=False,
333-         description="If True, exclude videos without an audio track (nosound=True)",
334-     )
335-
336-     # How matched videos are delivered: forward the video, send a link, or both.
337-     # Stored in the same JSON blob, so adding this field is backwards-compatible
338-     # (existing rows without it default to FORWARD).
339-     delivery_mode: DeliveryMode = Field(default=DeliveryMode.FORWARD)

```

11. assistant (2026-07-06T20:52:57.585Z)

Let me see the rest of FilterConfig and check for the model_config (extra keys handling), plus the tests.

12. user (2026-07-06T20:52:59.206Z)

```

339         delivery_mode: DeliveryMode = Field(default=DeliveryMode.FORWARD)
340
341         # Where matched videos / link messages are sent. Accepts:
342         #   - "me" (default): Saved Messages ? exact v1.1 behaviour.
343         #   - "@username": a public channel the account is a member of.
344         #   - "-100..." or a bare int-as-string: a numeric Telegram chat id
345         #     (private channels without a username).
346         # The string is passed verbatim to Telethon's forward_messages / send_message.
347         # See ADR 0005 for rationale and access requirements.
348         delivery_target: str = Field(
349             default="me",
350             min_length=1,
351             description="Telethon entity to deliver to: 'me', '@username', or a chat id",
352         )
353
354         @field_validator("delivery_target", mode="before")
355         @classmethod
356         def _coerce_delivery_target(cls, v: object) -> str:
357             """Accept int chat ids (e.g. -1001234567890) and coerce to str.
358
359             Telethon accepts both str and int as an entity, but storing as str keeps
360             the JSON blob uniform and lets '@username' / 'me' / numeric ids share
361             one field type.
362             """
363             if v is None:
364                 return "me"
365             if isinstance(v, int):
366                 return str(v)
367             return str(v)
368
369         @model_validator(mode="after")
370         def _check_duration_bounds(self) -> FilterConfig:
371             """Ensure min_duration_s <= max_duration_s when both are set."""
372             if (
373                 self.min_duration_s is not None
374                 and self.max_duration_s is not None
375                 and self.min_duration_s > self.max_duration_s
376             ):
377                 raise ValueError(
378                     f"min_duration_s ({self.min_duration_s}) must be <= max_duration_s "
379                     f"({self.max_duration_s})"

```

13. user (2026-07-06T20:52:59.391Z)

```

1      """Tests for ADR 0012 policy pipeline primitives."""
2
3      from __future__ import annotations
4
5      from tg_video_filter.models import ContentItem, Policy, Source
6      from tg_video_filter.policy import PolicyContext, StageResult, evaluate_pipeline
7
8
9      def _ctx() -> PolicyContext:
10         return PolicyContext(
11             content_item=ContentItem(
12                 account_id=111,
13                 source_id=1,
14                 provider_message_id="42",
15                 dedup_key="tg:msg:1:42",
16             ),
17             source=Source(
18                 id=1,
19                 account_id=111,
20                 provider="telegram",
21                 provider_source_id="-1001",
22             ),
23             policy=Policy(
24                 id=1,
25                 account_id=111,
26                 name="default",
27                 config_json={},
28                 is_default=True,
29             ),
30         )
31
32
33     class RecordingStage:
34         def __init__(
35             self,
36             name: str,
37             calls: list[str],
38             *,
39             passed: bool = True,
40             reason: str | None = None,
41         ) -> None:
42             self.name = name
43             self.calls = calls
44             self.passed = passed
45             self.reason = reason or ("passed" if passed else "failed")
46
47         async def evaluate(self, ctx: PolicyContext) -> StageResult:
48             self.calls.append(self.name)
49             return StageResult(
50                 passed=self.passed,
51                 reason=self.reason,
52                 stage_name=self.name,
53                 metadata={"content_item_id": ctx.content_item.id},
54             )
55
56
57     async def test_evaluate_pipeline_passes_all_stages_in_order() -> None:
58         calls: list[str] = []
59         stages = [
60             RecordingStage("metadata", calls),
61             RecordingStage("topic", calls),
62         ]
63
64         passed, results = await evaluate_pipeline(_ctx(), stages)
65

```

```

66         assert passed is True
67         assert calls == ["metadata", "topic"]
68         assert [r.stage_name for r in results] == ["metadata", "topic"]
69
70
71     async def test_evaluate_pipeline_short_circuits_on_first_failure() -> None:
72         calls: list[str] = []
73         stages = [
74             RecordingStage("metadata", calls),
75             RecordingStage("topic", calls, passed=False, reason="excluded topic"),
76             RecordingStage("security", calls),
77         ]
78
79         passed, results = await evaluate_pipeline(_ctx(), stages)
80
81         assert passed is False
82         assert calls == ["metadata", "topic"]
83         assert [r.stage_name for r in results] == ["metadata", "topic"]
84         assert results[-1].reason == "excluded topic"
85
86
87     async def test_evaluate_pipeline_with_no_stages_passes_vacuously() -> None:
88         passed, results = await evaluate_pipeline(_ctx(), [])
89
90         assert passed is True
91         assert results == []
92
93
94     def test_stage_result_helpers_attach_metadata() -> None:
95         passed = StageResult.pass_("metadata", "ok", duration_s=60)
96         failed = StageResult.fail("topic", "excluded", topic="crypto")
97
98         assert passed.passed is True
99         assert passed.metadata == {"duration_s": 60}
100        assert failed.passed is False
101        assert failed.metadata == {"topic": "crypto"}
102

```

14. assistant (2026-07-06T20:53:04.254Z)

Now let me check the FilterConfig model_config for extra keys handling ? this is critical for the merge. If FilterConfig ignores extra keys, load_filter_config's model_validate will silently drop unknown keys (which is fine, it's the flat view), but I need to verify the merge in save_filter_config truly preserves them. Let me also check whether `model\_config` has `extra="forbid"` which would break load_filter_config.

15. user (2026-07-06T20:53:06.105Z)

```

14:from pydantic import BaseModel, ConfigDict, Field, field_validator, model_validator
44:    model_config = ConfigDict(frozen=True)
65:    model_config = ConfigDict(frozen=True)
88:    model_config = ConfigDict(frozen=True)
104:    model_config = ConfigDict(frozen=True)
131:    model_config = ConfigDict(frozen=True)
159:    model_config = ConfigDict(frozen=True)
196:    model_config = ConfigDict(frozen=True)
230:    model_config = ConfigDict(frozen=True)
253:    model_config = ConfigDict(frozen=True)
285:    model_config = ConfigDict(frozen=True)
391:    model_config = ConfigDict(frozen=True)
402:    # Metadata extracted from the Telethon message (may be None if absent).

```

16. assistant (2026-07-06T20:53:08.962Z)

FilterConfig at line 299 does not appear to have a model_config in the snippet I read (lines 299-339). Let me check line 391 which is after FilterConfig ? it may belong to a different class. Let me confirm FilterConfig's config and whether extra keys are allowed on validation.

17. user (2026-07-06T20:53:10.027Z)

```
378             f"min_duration_s ({self.min_duration_s}) must be <= max_duration_s
({self.max_duration_s})"
379         )
380         return self
381
382
383     class VideoRecord(BaseModel):
384         """A seen video message + its metadata. Drives dedup and future /search.
385
386         Deprecated: a provider-neutral view over :class:`ContentItem` (ADR 0011).
387         New code should use :class:`ContentItem`; the ``db`` layer maps
388         ``VideoRecord`` ? ``ContentItem`` so existing callers and tests keep working.
389         """
390
391         model_config = ConfigDict(frozen=True)
392
393         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
394         user_id: int
395         channel_id: int = Field(description="Origin Telegram channel id")
396         message_id: int = Field(description="Telegram message id within the channel")
397         document_id: int | None = Field()
```

18. assistant (2026-07-06T20:53:15.822Z)

Confirmed: FilterConfig (line 299) has NO `model\_config`, so it uses Pydantic's default `extra="ignore"` ? extra keys are silently dropped on validation (fine for the flat view) and `model\_dump` won't emit them. The merge in save_filter_config relies on preserving the raw dict's unknown keys separately, which it does via `raw\_config.update(...)`.

Now let me run repros to test the key edge cases. Let me write a repro script covering: (1) the round-trip merge preserving unknown keys, (2) the `FilterConfig().model\_dump(mode="json")` default fallback in load_policy_config vs what evaluate does, (3) a critical asymmetry ? `load\_policy\_config` returns `FilterConfig().model\_dump(mode="json")` when no policy exists, which includes flat defaults, and (4) frozen dataclass metadata mutability.

19. assistant (2026-07-06T20:53:31.448Z)

```
import asyncio
import json
import aiosqlite
from datetime import UTC, datetime

from tg_video_filter import db
from tg_video_filter.models import Account, FilterConfig
from tg_video_filter.policy import StageResult, PolicyContext

async def setup(conn):
    await db._apply_schema(conn)
    await db.upsert_account(conn, Account(id=1, telegram_id=1, created_at=datetime.now(UTC)))

async def main():
    async with aiosqlite.connect(":memory:") as conn:
        await setup(conn)

        # ---- Seed a policy config with unknown pipeline keys ----
        await db.save_policy_config(conn, 1, {
```



```

        "min_duration_s": 5,
        "topics": ["crypto"],
        "security_checks": {"malware": True},
        "deep_inspect": True,
    })
    raw = await db.load_policy_config(conn, 1)
    print("1. after save_policy_config, load_policy_config:", raw)

    # ---- Now a legacy /set flips a flat field ----
    cfg = await db.load_filter_config(conn, 1)
    print("2. load_filter_config (flat view):", cfg.model_dump(mode="json").get("topics",
"NO topics key"))
    cfg2 = cfg.model_copy(update={"min_duration_s": 99})
    await db.save_filter_config(conn, 1, cfg2)
    raw2 = await db.load_policy_config(conn, 1)
    print("3. after save_filter_config, unknown keys preserved?:",
        "topics" in raw2, raw2.get("topics"), raw2.get("security_checks"),
raw2.get("deep_inspect"),
        "| min_duration_s=", raw2.get("min_duration_s"))

    # ---- ISSUE A: load_policy_config default fallback shape ----
    async with aiosqlite.connect(":memory:") as c2:
        await setup(c2)
        empty = await db.load_policy_config(c2, 1)
        print("4. load_policy_config with NO policy returns flat FilterConfig dump:",
            sorted(empty.keys()))

    # ---- ISSUE B: config that is valid JSON but not an object crashes load_filter_config
    ----
    # Simulate a policy row whose config_json is a JSON array/string (not object)
    await db.upsert_policy(conn, __import__("tg_video_filter.models",
fromlist=["Policy"]).Policy(
        id=None, account_id=2, name="default", config_json="[1,2,3]", is_default=True,
        created_at=datetime.now(UTC), updated_at=datetime.now(UTC),
    ))
    await db.upsert_account(conn, Account(id=2, telegram_id=2,
created_at=datetime.now(UTC)))
    # re-add policy for acct 2 properly
    try:
        await db.load_filter_config(conn, 2)
        print("5. load_filter_config non-object: no error (unexpected)")
    except Exception as e:
        print("5. load_filter_config with non-object config_json raises:", type(e).__name__,
e)

    # ---- ISSUE C: frozen dataclass with mutable metadata dict ----
    r = StageResult.pass_("s", "ok")
    r2 = StageResult.pass_("s", "ok")
    print("6. two pass_ results share metadata dict identity?:", r.metadata is r2.metadata)
    # metadata is mutable even though dataclass is frozen
    r.metadata["injected"] = 1
    print("7. frozen dataclass metadata mutated in place:", r.metadata)

    # default_factory: does empty-metadata default share?
    a = StageResult(passed=True, reason="x", stage_name="a")
    b = StageResult(passed=True, reason="y", stage_name="b")
    print("8. empty-default metadata shared identity?:", a.metadata is b.metadata)

```

```

asyncio.run(main())

```

20. user (2026-07-06T20:53:31.448Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

21. user (2026-07-06T20:53:35.912Z)

(Bash completed with no output)

22. assistant (2026-07-06T20:53:48.105Z)

```
import asyncio
import aiosqlite
from datetime import UTC, datetime

from tg_video_filter import db
from tg_video_filter.models import Account, FilterConfig, Policy
from tg_video_filter.policy import StageResult

async def setup(conn, acct_id=1):
    await db._apply_schema(conn)
    await db.upsert_account(conn, Account(id=acct_id, telegram_id=acct_id,
created_at=datetime.now(UTC)))

async def main():
    async with aiosqlite.connect(":memory:") as conn:
        await setup(conn)

        await db.save_policy_config(conn, 1, {
            "min_duration_s": 5,
            "topics": ["crypto"],
            "security_checks": {"malware": True},
            "deep_inspect": True,
        })
        raw = await db.load_policy_config(conn, 1)
        print("1. after save_policy_config:", raw)

        cfg = await db.load_filter_config(conn, 1)
        cfg2 = cfg.model_copy(update={"min_duration_s": 99})
        await db.save_filter_config(conn, 1, cfg2)
        raw2 = await db.load_policy_config(conn, 1)
        print("3. unknown keys preserved?:",
            "topics" in raw2, raw2.get("topics"), raw2.get("security_checks"),
            raw2.get("deep_inspect"), "| min_duration_s=", raw2.get("min_duration_s"))

        async with aiosqlite.connect(":memory:") as c2:
            await setup(c2)
            empty = await db.load_policy_config(c2, 1)
            print("4. load_policy_config NO policy keys:", sorted(empty.keys()))

        # non-object config_json
        await db.upsert_account(conn, Account(id=2, telegram_id=2,
created_at=datetime.now(UTC)))
        await db.upsert_policy(conn, Policy(
            id=None, account_id=2, name="default", config_json="[1,2,3]", is_default=True,
            created_at=datetime.now(UTC), updated_at=datetime.now(UTC),
        ))
        try:
            r = await db.load_filter_config(conn, 2)
            print("5. load_filter_config non-object: NO error:", r)
        except Exception as e:
            print("5. load_filter_config non-object RAISES:", type(e).__name__, str(e)[:80])

        r1 = StageResult.pass_("s", "ok")
        r2 = StageResult.pass_("s", "ok")
        print("6. two pass_ share metadata identity?:", r1.metadata is r2.metadata)
        r1.metadata["injected"] = 1
        print("7. frozen dataclass metadata mutated in place:", r1.metadata)
```

```

a = StageResult(passed=True, reason="x", stage_name="a")
b = StageResult(passed=True, reason="y", stage_name="b")
print("8. empty-default metadata shared identity?:", a.metadata is b.metadata)

```

```

asyncio.run(main())

```

23. user (2026-07-06T20:53:48.147Z)

File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/repro.py (file state is current in your context ? no need to Read it back)

24. user (2026-07-06T20:53:53.081Z)

1. after save_policy_config: {'deep_inspect': True, 'min_duration_s': 5, 'security_checks': {'malware': True}, 'topics': ['crypto']}
3. unknown keys preserved?: True ['crypto'] {'malware': True} True | min_duration_s= 99
4. load_policy_config NO policy keys: ['delivery_mode', 'delivery_target', 'exclude_gif', 'exclude_nosound', 'exclude_round', 'max_duration_s', 'max_size_mb', 'min_duration_s', 'target_height']
5. load_filter_config non-object RAISES: ValueError policy config_json must decode to a JSON object
6. two pass_ share metadata identity?: False
7. frozen dataclass metadata mutated in place: {'injected': 1}
8. empty-default metadata shared identity?: False

25. assistant (2026-07-06T20:54:01.370Z)

Key results:

- Merge preserves unknown keys correctly (happy path confirmed).
- `pass\_`/`fail` do NOT share metadata dict (each gets fresh `\*\*kwargs`), and the `default\_factory=dict` gives fresh dicts too (result 8) ? so no mutable-default bug.
- Frozen dataclass metadata IS mutable in place (result 7) ? a frozen/mutability inconsistency, low severity.
- **Result 5 is interesting**: `load\_filter\_config` now raises `ValueError` on a non-object config_json, whereas the OLD code used `model\_validate\_json` which would ALSO raise (validation error). So not a regression per se. Let me verify the old behavior to be precise.

Let me now investigate the more important concern: the asymmetry in `load\_policy\_config`'s empty fallback (result 4). It returns a FLAT FilterConfig dump, but `save\_policy\_config` writes an arbitrary raw dict. Let me check how these are meant to be consumed and check the old `model\_validate\_json` behavior on non-object, plus test the round-trip when NO policy exists yet through save_filter_config.

26. user (2026-07-06T20:54:06.326Z)

```

old model_validate_json([1,2,3]) RAISES: ValidationError
model_validate ignores extras: 5

```

27. assistant (2026-07-06T20:54:09.489Z)

Both old and new raise on non-object JSON, so no regression there. Now let me check `Iterable` import (I saw it imported at line 15) ? is it still used? Actually that's pre-existing. Let me focus on the callers of these functions and whether policy.py is truly unreachable.

28. user (2026-07-06T20:54:10.787Z)

```

C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\policy.py:55:class PolicyStage(Protocol):
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\policy.py:63:async def evaluate_pipeline(
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\policy.py:65:    stages: list[PolicyStage],
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\policy.py:79:    "PolicyStage",
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\policy.py:81:    "evaluate_pipeline",
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\db.py:1310:async def

```

```
load_policy_config(conn: aiosqlite.Connection, account_id: int) -> dict[str, object]:
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\db.py:1323:async def save_policy_config(
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\db.py:1778:     await
save_policy_config(conn, user_id, raw_config)
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\db.py:1845:     "load_policy_config",
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\db.py:1846:     "save_policy_config",
```

29. assistant (2026-07-06T20:54:18.531Z)

Confirmed: `evaluate\_pipeline`, `PolicyStage`, `PolicyContext`, `StageResult` are NOT called anywhere in `src/` ? only in tests. `policy.py` is entirely unreachable production code in this slice. `load\_policy\_config`/`save\_policy\_config` also have no production callers yet (only `save\_filter\_config` calls `save\_policy\_config` internally). This is disclosed as "first F1 slice" so it's a low-severity scope note.

Now let me check one important asymmetry I want to nail down: `load\_policy\_config`'s empty fallback returns a flat `FilterConfig().model\_dump(mode="json")`, but `save\_policy\_config` writes the raw dict as-is. And `\_policy\_config\_dict\_to\_json` uses `sort\_keys=True`. Let me check whether there's a concern with the `save\_filter\_config` merge when a stored config has a key that FilterConfig would normalize differently. Let me verify one concrete data-integrity concern: does the merge re-emit ALL flat FilterConfig fields even ones the user never set, potentially resurrecting defaults? Let me trace a scenario.

30. user (2026-07-06T20:54:25.715Z)

```
loaded flat min_duration_s= 3 delivery_target= me
after /set, stored keys: ['delivery_mode', 'delivery_target', 'exclude_gif', 'exclude_nosound',
'exclude_round', 'max_duration_s', 'max_size_mb', 'min_duration_s', 'target_height', 'topics']
exclude_gif now present (default resurrected)? True
```

31. assistant (2026-07-06T20:54:34.575Z)

This confirms behavior but it's expected/benign: `save\_filter\_config` always writes the full flat FilterConfig (all fields with their effective values, which for unset ones = defaults). This matches the OLD behavior too (old code did `cfg.model\_dump\_json()`) which also wrote all fields). So no regression ? the merge just adds the unknown `topics` key on top. Good.

Now let me nail down the one genuine data-loss edge case I'm suspicious about: what happens in `save\_filter\_config` when `existing.config\_json` is valid JSON but a non-dict (e.g. `"null"` or `""`)? The code catches `(json.JSONDecodeError, ValueError)` and resets `raw\_config = {}`. That means unknown keys would be LOST ? but only if the stored config was already malformed (non-object), which shouldn't happen through normal writes. Let me verify `\_policy\_config\_json\_to\_dict` raises `ValueError` (caught) vs something else, and check the `json.loads("null")` case.

32. user (2026-07-06T20:54:36.312Z)

```
'null' -> RAISES ValueError
'[]' -> RAISES ValueError
'"str"' -> RAISES ValueError
'123' -> RAISES ValueError
'{bad}' -> RAISES JSONDecodeError
'' -> RAISES JSONDecodeError
```

33. assistant (2026-07-06T20:54:43.248Z)

Both exception types are caught in `save\_filter\_config`. So a corrupt stored config degrades gracefully to `{}` (flat rewrite) rather than crashing. Reasonable.

But note the asymmetry: `load\_filter\_config` (line 1759) calls `\_policy\_config\_json\_to\_dict` **without** a try/except, so it will **raise** on a corrupt/non-object stored config, while `save\_filter\_config` swallows it. And `load\_policy\_config` (line 1320) also raises. This is a latent asymmetry: a config row that `save\_filter\_config` would silently "heal" is one that `load\_filter\_config`/`load\_policy\_config` would crash on. In practice the stored config is

always an object, so this is low severity, but it's an inconsistency worth noting.

Now let me verify the async correctness of `evaluate\_pipeline` and confirm the `Iterable` import is pre-existing (not this PR). Let me also confirm there's no issue with `StageResult` being frozen but `pass\_`/`fail` being `@staticmethod` returning the class - and check the `object | None` typing gap for `fetcher/budget` against the ADR.

34. user (2026-07-06T20:54:46.447Z)

```
14:from collections.abc import Awaitable, Callable, Iterable
---ADR fetcher/budget refs---
67:?      ? gated behind explicit user opt-in + budget      ?
83:  fetcher: ContentFetcher | None = None # only present for opt-in I/O stages
84:  budget: PolicyBudget | None = None    # bytes/time/concurrency limits for I/O
98:`ContentFetcher` is an implementation detail introduced in F5, not a
99:requirement for metadata-only stages. F1-F4 must work when `ctx.fetcher` is
101:`ctx.budget`.
162:`config_json`) and the stage has a **budget** (max bytes per item, max
176:- The stage receives a bounded `ContentFetcher` and an enforced
177: `PolicyBudget`.
199:The caller (Telethon userbot path or future Bot API handler) uses the final
231:| **F1** ? Policy engine prototype | `PolicyStage` protocol, `evaluate_pipeline()`,
raw/typed policy-config persistence, and hot-path wiring from live handler to
`ContentItem`/`Source`/`Policy` context | Tracks B-E landed |
235:| **F5** ? Deep inspection | `ContentFetcher`, fprobe/perceptual hash, download budget
enforcement | F1, accepted ADR 0003 amendment above |
260: (metadata-only vs. I/O-heavy). Mitigated by keeping F1-F4 fetcher-free and
261: making Tier 4's fetcher/budget explicit.
270:- Deep inspection amends ADR 0003 and must be explicitly opt-in with a budget
280: ones without touching the hot path.
299:4. **Deep inspection budget** ? per-item byte limit or per-account
```

35. assistant (2026-07-06T20:54:53.264Z)

The ADR confirms two things:

1. The ADR names `ContentFetcher`/`PolicyBudget` as the intended types for `fetcher`/`budget` (line 83-84), but PR uses `object | None`. `ContentFetcher` doesn't exist until F5, so `object | None` is a deliberate placeholder ? acceptable but a typing gap (low). The ADR explicitly says F1-F4 must work with `ctx.fetcher` is `None`, which the default `None` satisfies.
2. ****The ADR's F1 scope** (line 231) explicitly includes "hot-path wiring from live handler to `ContentItem`/`Source`/`Policy` context".** This PR does NOT touch `telethon\_client.py` ? `evaluate\_pipeline`/`PolicyStage` are called nowhere in production. So this PR delivers a **subset** of ADR-defined F1. The PR body says "first F1 slice," so it's disclosed. Low-severity scope note.

Let me verify `Iterable` is used elsewhere (pre-existing, not this PR's concern) and do a final check on whether there are any existing production callers of `load\_filter\_config`/`save\_filter\_config` that would be affected by the merge change.

36. user (2026-07-06T20:54:54.325Z)

```
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\db.py:619:      #      (Option 2A).
is_default=1 so the legacy load_filter_config shim reads
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\db.py:1313:    Unlike
:func:`load_filter_config`, this preserves future ADR 0012 keys such
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\db.py:1749:async def
load_filter_config(conn: aiosqlite.Connection, user_id: int) -> FilterConfig:
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\db.py:1762:async def
save_filter_config(conn: aiosqlite.Connection, user_id: int, cfg: FilterConfig) -> None:
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\db.py:1864:    "load_filter_config",
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\db.py:1865:    "save_filter_config",
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\manager.py:140:        cfg = await
```

```

db.load_filter_config(conn, session.account_id)
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\models.py:193:    ``load_filter_config`` /
``save_filter_config`` shims read/write that row.
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\command_service.py:465:        await
db.save_filter_config(conn, user_id, cfg)
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\commands.py:78:        current_cfg = await
db.load_filter_config(conn, user_id)
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\commands.py:82:        new_cfg = await
db.load_filter_config(conn, user_id)
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\commands.py:112:    cfg = await
db.load_filter_config(conn, user_id)
C:\Users\avidu\Projects\wt-pr50\src\tg_video_filter\bootstrap.py:75:        await
db.save_filter_config(conn, user_id, DEFAULT_FILTER_CONFIG)

```

37. assistant (2026-07-06T20:54:59.615Z)

The `command\_service.py:465` is the `/set` path (the one the PR is designed to protect).
`bootstrap.py:75` writes `DEFAULT\_FILTER\_CONFIG` on account creation. These are exercised and
the happy path was verified by the main reviewer.

Let me verify one last concrete concern in the `command\_service.py` `/set` flow ? the pattern is
load?modify?save. There's a potential concurrency/lost-update window but that's pre-existing
(read-modify-write on flat config always had it). Let me confirm the `/set` implementation
doesn't do anything that would defeat the merge.

38. user (2026-07-06T20:55:00.486Z)

```

440         max_size_mb: int | None = None,
441         max_duration_s: int | None = None,
442         min_duration_s: int | None = None,
443         target_height: int | None = None,
444         exclude_gif: bool | None = None,
445         exclude_round: bool | None = None,
446         exclude_nosound: bool | None = None,
447     ) -> FilterConfig:
448         """Apply partial updates to the account's config, persist, and return."""
449         updates: dict[str, object] = {}
450         if max_size_mb is not None:
451             updates["max_size_mb"] = max_size_mb
452         if max_duration_s is not None:
453             updates["max_duration_s"] = max_duration_s
454         if min_duration_s is not None:
455             updates["min_duration_s"] = min_duration_s
456         if target_height is not None:
457             updates["target_height"] = target_height
458         if exclude_gif is not None:
459             updates["exclude_gif"] = exclude_gif
460         if exclude_round is not None:
461             updates["exclude_round"] = exclude_round
462         if exclude_nosound is not None:
463             updates["exclude_nosound"] = exclude_nosound
464         cfg = current.model_copy(update=updates) if updates else current
465         await db.save_filter_config(conn, user_id, cfg)
466         log.info(
467             "config_updated user=%s max_size_mb=%s min_duration_s=%s max_duration_s=%s "
468             "target_height=%s exclude_gif=%s exclude_round=%s exclude_nosound=%s",
469             user_id,

```

39. assistant (2026-07-06T20:55:06.471Z)

The `/set` flow is a clean load?model_copy?save. The merge in `save\_filter\_config` correctly
preserves unknown keys through this path (verified in repro result 3).

I have thoroughly investigated all dimensions. Let me consolidate findings. The primitives are

correct: no short-circuit bug (appends failing result then returns ? confirmed by tests and code), no mutable-default bug (each `pass\_`/`fail` gets fresh `\*\*metadata`, `default\_factory=dict` gives fresh dicts), async is correct. The findings are edge-case/scope observations, all low-to-medium severity ? the authoritative gate already passed and the happy path works.

40. user (2026-07-06T20:55:37.535Z)

Structured output provided successfully